

Plataforma de E-commerce

Modelo de dados para cadastro, catálogo, pedidos, pagamentos, estoque, cupons, logística e avaliações.

Integrantes

Sofia Ferreira Costa
Francisco Maycon Lima Castro
Rafael Henrique de Oliveira Cruz
Thales de Souza Ferreira
Thiago Junio

Fluxo central

01

Cliente

cadastro, endereço e contato

02

Pedido

itens, cupom, status e valor

03

Pagamento

cartão, Pix ou boleto

04

Operação

estoque, logística e avaliação

O que o banco precisa controlar?

O modelo cobre a jornada completa da loja, do cadastro ao pós-venda.

Cadastro

Cliente, CPF, e-mail único, endereço, CEP normalizado e múltiplos telefones.

Catálogo

Categoria, produto, preço, comparação de preço, disponibilidade e estoque.

Venda

Pedido centraliza a compra; Item_Pedido registra produtos, quantidade e preço histórico.

Financeiro

Pagamento comum com dados específicos separados em cartão, Pix e boleto.

Operação

Cupom, controle de estoque, rastreamento e atualização da entrega.

Pós-venda

Avaliação de produto por cliente, com nota entre 1 e 5 e sem duplicidade.

Regras de negócio

As decisões de modelagem partem das regras que mantêm o fluxo comercial coerente.

01

Cliente antes do pedido

Todo pedido pertence a um único cliente existente.

02

Pedido com produtos

Os produtos entram por Item_Pedido, nunca diretamente em Pedido.

03

Preço preservado

O item guarda o preço praticado na compra, mesmo se o catálogo mudar.

04

Pagamento específico

Cada pagamento usa cartão, Pix ou boleto e mantém apenas os campos do método.

05

Cupom e logística opcionais

O pedido pode nascer sem desconto e receber a entrega depois.

06

Avaliação controlada

O cliente avalia cada produto uma vez, com nota válida de 1 a 5.

07

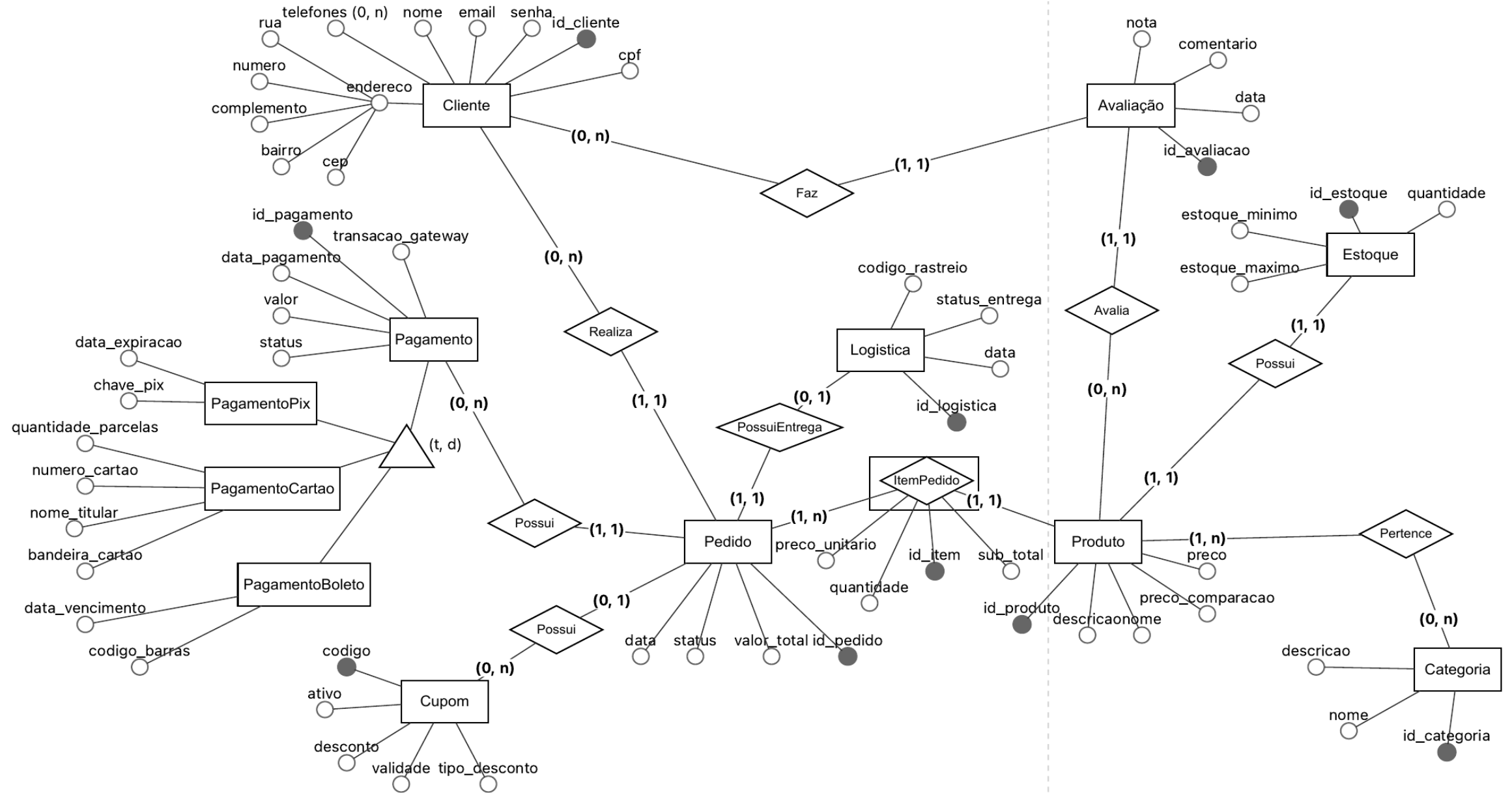
Estoque consistente

Quantidade não pode ser negativa nem ultrapassar o limite máximo.

08

Status válidos

Pedido, pagamento e entrega aceitam somente estados previstos.



Do DER ao modelo lógico

A transformação preserva as relações e ajusta o que precisa virar tabela, chave ou cálculo.

No DER	No modelo relacional
Telefones multivalorados	cliente_telefone(id_cliente, telefone)
Pedido × Produto	item_pedido resolve a relação M:N
CEP no endereço	cep separa bairro, cidade e UF
Produto × Estoque 1:1	estoque.id_produto é PK e FK
Especialização Pagamento	Tabela pai + cartão, Pix e boleto
Subtotal do item	Calculado por quantidade × preco_unitario

Sem FK redundante

Ex: Pedido não recebe id_produto; a associação pertence a Item_Pedido.

Especialização sem campos vazios

Ex: Dados comuns ficam em Pagamento; campos de cada método ficam no subtipo.

Histórico preservado

Ex: O preço unitário da compra não depende do preço atual do produto.

Arquitetura lógica: 15 tabelas

As tabelas são agrupadas por responsabilidade para manter o modelo legível e evolutivo.

Cadastro

cep

cliente

cliente_telefone

Dados pessoais,
endereço e contato.

Catálogo

categoria

produto

estoque

Oferta comercial e
disponibilidade.

Venda

pedido

item_pedido

cupom

logistica

Compra, desconto e
entrega.

Financeiro

pagamento

pagamento_cartao

pagamento_pix

pagamento_boleto

Processamento e
métodos de pagamento.

Pós-venda

avaliacao

Nota, comentário e data
por cliente/produto.

Reduzir repetição e evitar inconsistência

As dependências funcionais orientam o que deve ser separado, mantido ou calculado.

Tabela	Dependência observada	Decisão	Resultado
Cliente	id_cliente → cep → bairro/cidade/UF	Separar dados do CEP	cliente + cep
Item_Pedido	quantidade + preço → subtotal	Não armazenar subtotal	Cálculo feito na consulta
Produto	id_produto → atributos	Sem dependência transitiva	Tabela permanece estável

1FN

Atributos atômicos;
telefone vira tabela
própria.

2FN

Sem dependências
parciais indevidas.

3FN

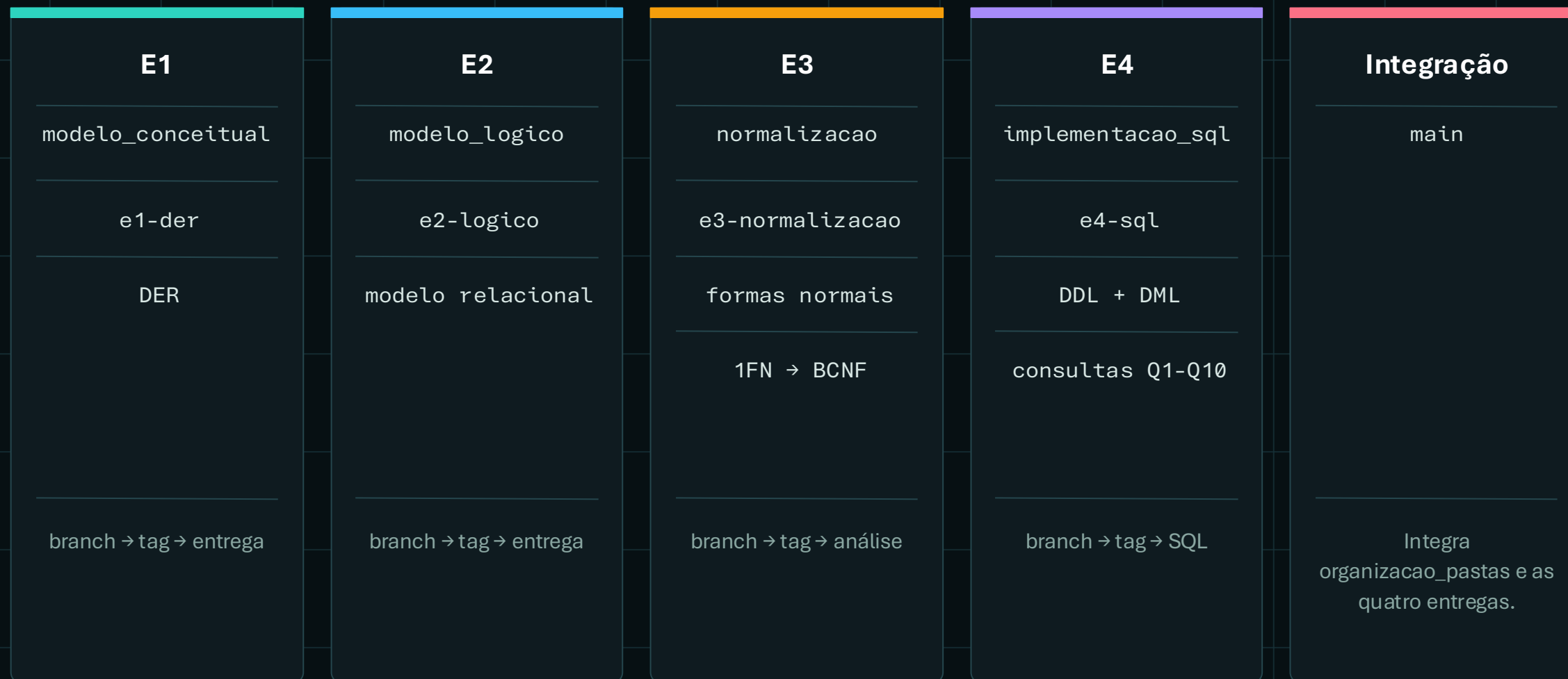
CEP remove
dependência transitiva.

BCNF

Determinantes
relevantes são chaves
candidatas.

Branches e tags por etapa

Branches isolaram as etapas; tags registraram E1-E4 e a main concentrou a versão final.



O que o PostgreSQL garante

Constraints protegem os dados; regras de conjunto exigem aplicação ou trigger.

PK

Identidade única de cada registro.

FK

Relacionamentos apontam para registros existentes.

UNIQUE

E-mail, CPF, pagamento por pedido e avaliação sem duplicidade.

CHECK

Preços, notas, quantidades e status válidos.

NOT NULL

Campos obrigatórios não ficam vazios.

CASCADE

Dependências selecionadas acompanham a exclusão.

Regra	Cobertura
Item referencia pedido e produto existentes	Banco
No máximo um pagamento por pedido	Banco
Nota entre 1 e 5	Banco
Pedido deve possuir ao menos um item	Aplicação / trigger
Todo produto deve possuir estoque	Aplicação / trigger
Pagamento deve ter exatamente um subtipo	Aplicação / trigger

Dados de teste com fluxo completo

A massa de dados conecta cadastro, venda, pagamento, entrega e pós-venda.

10

clientes

10

produtos

10

pedidos

20

itens de pedido

10

pagamentos

10

avaliações

Pedido 1: cálculo verificável

Smartphone Orion 128GB × 1	R\$ 2.499,90
----------------------------	--------------

Controle Pro Wireless × 1	R\$ 349,90
---------------------------	------------

Subtotal dos itens	R\$ 2.849,80
--------------------	--------------

Cupom BEMVINDO10	- 10%
------------------	-------

R\$ 2.564,82

Valor gravado em pedido.valor_total, exatamente igual ao subtotal com desconto.

Pedido e itens em funcionamento

Os registros conectam a compra aos produtos sem repetir dados do catálogo.

Tabela pedido

id	cliente	cupom	status	valor
1	1	BEMVINDO10	entregue	2564.82
2	2	NULL	enviado	5198.90
3	3	GAMES25	pago	504.70
10	10	PULSE10	criado	836.73

Tabela item_pedido

pedido	produto	qtd	preço_unit.
1	1	1	2499.90
1	3	1	349.90
2	2	1	4599.00
2	4	1	599.90

Leitura do exemplo

O pedido 1 pertence ao cliente 1 e possui dois produtos. Cada item usa a chave composta (id_pedido, id_produto).

Tabela associativa de itens

A implementação resolve Pedido × Produto e leva as regras diretamente ao SQL.

schema.sql / item_pedido

```
CREATE TABLE item_pedido (  
  id_pedido INTEGER NOT NULL,  
  id_produto INTEGER NOT NULL,  
  quantidade INTEGER NOT NULL CHECK (quantidade > 0),  
  preco_unitario NUMERIC(10,2) NOT NULL  
    CHECK (preco_unitario > 0),  
  
  PRIMARY KEY (id_pedido, id_produto),  
  
  CONSTRAINT fk_item_pedido_pedido  
    FOREIGN KEY (id_pedido)  
    REFERENCES pedido(id_pedido)  
    ON DELETE CASCADE,  
  
  CONSTRAINT fk_item_pedido_produto  
    FOREIGN KEY (id_produto)  
    REFERENCES produto(id_produto)  
);
```

Chave composta

Identifica cada produto dentro de um pedido.

Duas FKs

Garante pedido e produto existentes.

Valores válidos

Quantidade e preço precisam ser positivos.

Q1 a Q10 respondem perguntas reais

As consultas demonstram filtros, junções, agregações, subconsultas, view e decisão.

Filtros

Q1 · Q2

Produtos por preço e clientes por cidade, domínio de e-mail e inicial do nome.

Junções

Q3 · Q4

Pedidos por cliente e produtos por pedido, com categoria e subtotal calculado.

LEFT JOIN

Q5

Produtos que ainda não possuem nenhuma avaliação.

Agregação e view

Q6 · Q9

Receita por categoria e resumo de quantidade/valor comprado por cliente.

Subconsultas

Q7 · Q8

Produtos e pedidos acima de médias calculadas a partir dos próprios dados.

Decisão

Q10

Prioridade de reposição por estoque baixo, unidades vendidas e receita gerada.

Q10: prioridade de reposição

O modelo combina estoque atual e histórico de venda para apoiar uma decisão operacional.

● ● ● Q10 / consulta

```
SELECT pr.id_produto, pr.nome,
       es.quantidade, es.estoque_minimo,
       COALESCE(SUM(CASE WHEN p.id_pedido IS NOT NULL
                        THEN ip.quantidade ELSE 0 END), 0) AS unidades_vendidas,
       COALESCE(SUM(CASE WHEN p.id_pedido IS NOT NULL
                        THEN ip.quantidade * ip.preco_unitario ELSE 0 END), 0)
       AS receita_gerada
FROM produto pr
JOIN estoque es ON es.id_produto = pr.id_produto
LEFT JOIN item_pedido ip ON ip.id_produto = pr.id_produto
LEFT JOIN pedido p ON p.id_pedido = ip.id_pedido
  AND p.status <> 'cancelado'
WHERE es.quantidade <= es.estoque_minimo
GROUP BY pr.id_produto, pr.nome,
         es.quantidade, es.estoque_minimo
ORDER BY receita_gerada DESC, unidades_vendidas DESC;
```

Camera Action
4K

produto priorizado

4 / 5

estoque atual / mínimo

1

unidade vendida

R\$ 1.199,90

receita gerada

Resultado sobre os dados do projeto

A câmera é o único produto abaixo do estoque mínimo e aparece primeiro para reposição. Pedidos cancelados não entram no histórico.

Resultado técnico do projeto

O modelo conecta conceito, estrutura relacional, integridade e uso prático dos dados.

Modelo completo

Cliente, catálogo, pedido, pagamento, estoque, cupom, logística e avaliação.

Conceitos aplicados

DER, cardinalidade, M:N, especialização, normalização, PK, FK, UNIQUE e CHECK.

Implementação demonstrável

DDL, DML, dados coerentes, documentação e consultas Q1 a Q10.

DER



Modelo lógico



Normalização



PostgreSQL

O banco sustenta cadastro, vendas, operação e análise em uma loja online.

FIM

OBRIGADO A TODOS!

Banco de Dados – Puc Minas - Lourdes